“Operating System Design and Implementation”

(Chapter 4)
File System and Storage Management

Contents

This Chapter will cover:

- Distributed File Systems: HDFS, Ceph, Google File System
- Storage Technologies: SSDs, NVM, Persistent Memory.
- Modern File Systems: ZFS, Btrfs, EXT4, APFS
- Scalability and Consistency Challenges in file and storage.
- Case Studies: DNA Storage, Holographic Storage, Quantum Storage

File System

Introduction to file system

- Files are **named to a collection** of related information that is recorded on the secondary storage i.e. data can't be written to secondary storage unless they are within file.
- The **OS abstracts from the physical properties** of its storage device to define a logical storage unit i.e. the file. Files are mapped by the operating system onto the physical devices.
- File system provides an abstract view of secondary storage and is responsible for **global naming, file access, and overall file organization**. These functions are handled by the name service, the file service, and the directory service.
- File service is the specification of what the file system offers to its clients. File server is a process that runs on some machine and helps implement the file service.

File System

File Types

- Every operating system supports several types of file.
- If an operating system recognizes the type of file, it can then operate on the file in reasonable way.
- When a file is created by any process, then it is giving a name.
- The name is split into two parts: **a filename and an extension separated by period (.).**
- The system uses the extension to indicate the type of file and type of operation that can be done by that particular file.
- **Whenever the user tries the open a saved file then, the operating system determines which program to run based on the file extension (type).**
- Major file types includes: Program File (e.g. demo.cpp), Compressed File(e.g. demo.zip), Graphics File (e.g. demo.jpg), Sound File (e.g. demo.mp3), Document File (e.g. demo.doc) etc.

File System

File Naming

- **Files are an abstraction mechanism**. They provide a way to store information on the storage medium i.e. disk and read it back later. This must be done in such a way as to shield the users from the detail of **how and where the information is stored, and how the disks actually work.**
- When a process creates a file, it gives the file a name. Files have unique file names and may be created and deleted.
- **Naming refers to the mapping between logical (file names and directory structures that users see and interact with) and physical objects(actual storage locations on disk).**
- The file system maintains a mapping between logical filenames and their physical storage through data structures such as **FAT , Inodes etc**.

For example, when a user opens document.txt:

1. The file system looks up the filename in the directory structure.
2. It retrieves metadata about the file (such as size, owner, and physical storage locations).
3. It translates the logical filename into actual disk blocks where the file's content is stored.
4. The file system reads the data from those blocks and presents it to the user.

Traditional File System

Introduction to FAT(File Allocation Table)

- The File Allocation Table (FAT) file system was developed by Microsoft in 1977 and has been widely used in early operating systems like MS-DOS and Windows.
- It is simple, efficient, and commonly used in portable storage devices like USB flash drives and memory cards.

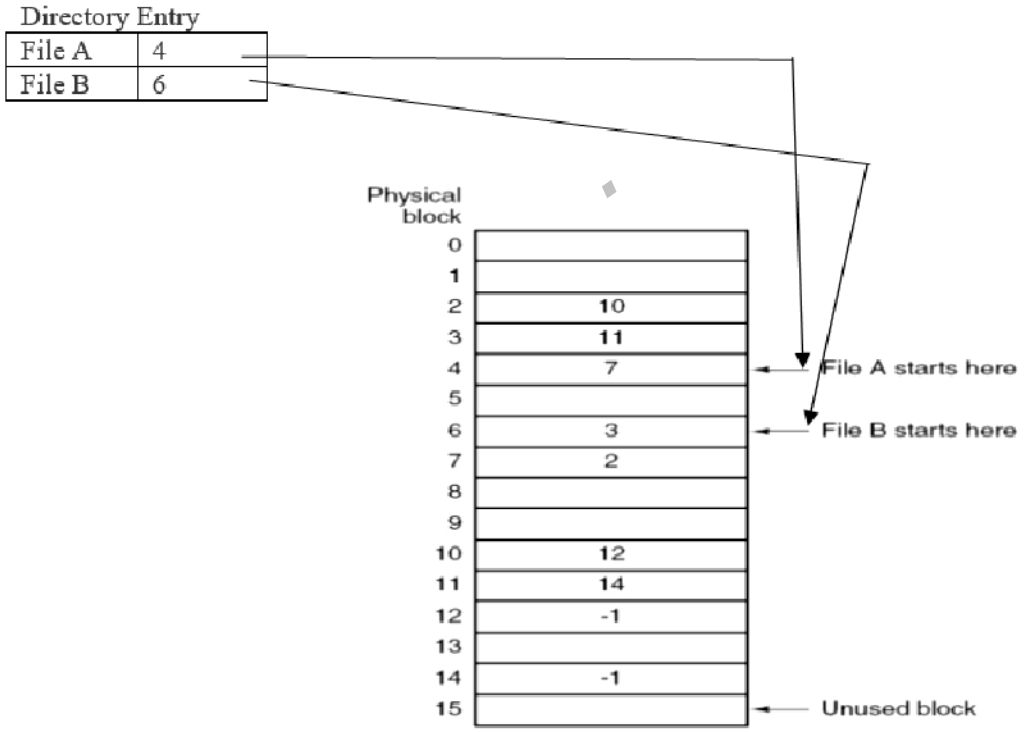


Fig: Linked list Allocation using a File Allocation Table in main memory

Traditional File System

Example: FAT(File Allocation Table)

- FAT is a simple and efficient method of disk-space allocation used in MS-DOS and Windows. It organizes files using a linked-list approach stored in a table in memory.
- A section at the beginning of the disk stores the File Allocation Table (FAT). FAT contains one entry per disk block, indexed by block number. Each entry stores the next block number in the file sequence or an end-of-file marker (-1). Unused blocks are marked with 0.
- The directory i.e. Root Directory, entry stores only the first block number of the file. Further directory also stores file and directory metadata like filename, file size, file attribute(read-only, hidden etc.), timestamp(creation, modified etc.). The FAT is traversed in memory to find subsequent blocks. This improves random access as no disk I/O is needed for block traversal.

Example:

- File A (stored in blocks 4 → 7 → 2 → 10 → 12)
- File B (stored in blocks 6 → 3 → 11 → 14)

The FAT table helps follow the chain from the starting block to the last block.

Advantages:

- Simple & Efficient – Easy to implement and manage.
- Supports Random Access – Since the chain is in memory, files can be accessed faster than a pure linked-list approach.

Traditional File System

Introduction to FAT(File Allocation Table)

Disadvantages:

- Large Memory Requirement – The FAT must be kept in RAM all the time.

  Example: A 1 TB disk with 1 KB blocks needs 1 billion entries.

  If each entry is 4 bytes, it consumes 4 GB RAM.
- Not Scalable – The system is inefficient for large disks due to memory usage.
- Fragmentation Issues – Files are scattered across different blocks, slowing access.
- Usage & Legacy: FAT was the original file system for MS-DOS and early Windows versions. It is still widely used in USB drives, memory cards, and embedded systems due to its simplicity and compatibility.

Other Traditional file system includes NTFS-New Technology File System(used in windows), ext-Extended File System (used in Linux), ISO 9660 (used for optical Disk) etc.

Modern File System

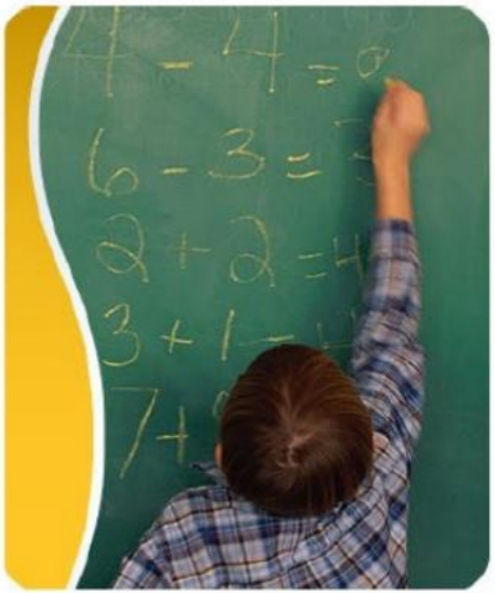
Introduction

- A file system is a fundamental component of any operating system, responsible for managing how data is stored and retrieved on storage devices.
- With growing data needs, file systems have evolved to offer **better performance, scalability, and reliability.**
- Modern file systems go beyond basic data storage, integrating advanced features like **snapshots, error correction, and efficient space management**.
- Next, we will explore two of the modern file systems: ZFS and Btrfs.

Modern File System

Introduction

Why a New File System?



Data Management Costs are High

The Value of Data is Becoming Even More Critical

The Amount of Storage is Ever-Increasing

Modern File System

Limitation of Older File System

Older file systems, while effective for their time, suffer from several limitations:

- **No Defense Against Silent Data Corruption:** Older file systems lack built-in mechanisms to detect and correct silent data corruption, leading to potential data loss. Modern file systems store checksums (hash values) alongside data. ZFS & Btrfs: Use end-to-end checksums to verify data integrity. APFS (Apple File System): Implements metadata checksumming. ZFS: Uses RAID-Z and self-healing to fix corrupt data. Btrfs: Can repair corrupted files if mirrored copies exist.
- **Difficult Administration:** Traditional file systems require manual management of volumes, partitions, and labels, adding complexity and administrative overhead. Modern file systems allow quick snapshots of the current system state that helps in Helps in backup, rollback, and disaster recovery. File systems like ZFS, Btrfs allows: Automatic resizing of volumes. On-the-fly partition adjustments without downtime.

Modern File System

Limitation of Older File System

- **Slower Data Management Techniques:**
  - ✓ Outdated techniques like fixed block sizes (leading wastage space due to fragmentation), fat locks(use coarse grained locking mechanisms which leads unnecessary contention as locks large sections of a database or file), and naive pre-fetching (loads data into memory before it is actually requested) reduce efficiency and performance.
  - ✓ Modern file systems use dynamic or variable block sizes to optimize storage and speed. Fine-grained locks (locking only the necessary data) improve performance and allow concurrent access without major slowdowns. Modern techniques use predictive algorithms (e.g., machine learning, access pattern analysis) to optimize pre-fetching and improve efficiency.

Modern File System

Limitation of Older File System

- **Lack of Snapshot and Versioning Support**
  - ✓ Older file systems do not support snapshots or versioning, making it difficult to restore previous states of files after accidental deletions or corruption.
  - ✓ Modern file systems like ZFS, Btrfs, and APFS allow instant snapshots, which help with backup, rollback, and disaster recovery.

- **Poor Scalability**
  - ✓ Older file systems struggle with handling large amounts of data efficiently.
  - ✓ Traditional file systems (e.g., ext3, FAT32) have limitations on maximum file size and volume size.
  - ✓ Newer systems like ZFS scale better by supporting multi-petabyte storage and high-speed parallel processing.

- **No Built-in Redundancy or Self-Healing**
  - ✓ Older file systems do not have mechanisms for self-healing or redundancy.
  - ✓ Corruption or disk failure can lead to irrecoverable data loss.
  - ✓ Modern systems like ZFS and Btrfs use RAID-Z, checksumming, and self-healing to detect and fix errors automatically.

Modern File System

Limitation of Older File System

- **Limited Security Features**
  - ✓ Older file systems lack advanced encryption, access control, and auditing.
  - ✓ For example, FAT32 and ext2 do not support native encryption.
  - ✓ Modern file systems like APFS and NTFS provide built-in encryption, journaling, and permission controls.
- **No Efficient Handling of SSDs**
  - ✓ Older file systems were designed for HDDs and do not optimize wear leveling and TRIM support for SSDs. Wear Leveling prevents early SSD failure by evenly distributing write cycles. Unlike HDDs, SSDs cannot overwrite existing data directly—they must erase blocks before writing new data. TRIM keeps SSDs fast by marking unused blocks for cleanup, reducing unnecessary writes.
  - ✓ Modern SSD-aware file systems (e.g., APFS and Btrfs) improve performance and lifespan of SSDs.

Modern File System

ZFS (Zettabyte File System)

- Originally developed by Sun Microsystems for Solaris, ZFS is now widely used in FreeBSD, Linux, and enterprise storage solutions.
- It is known for its robustness, high data integrity, and advanced storage management capabilities.

Key Features

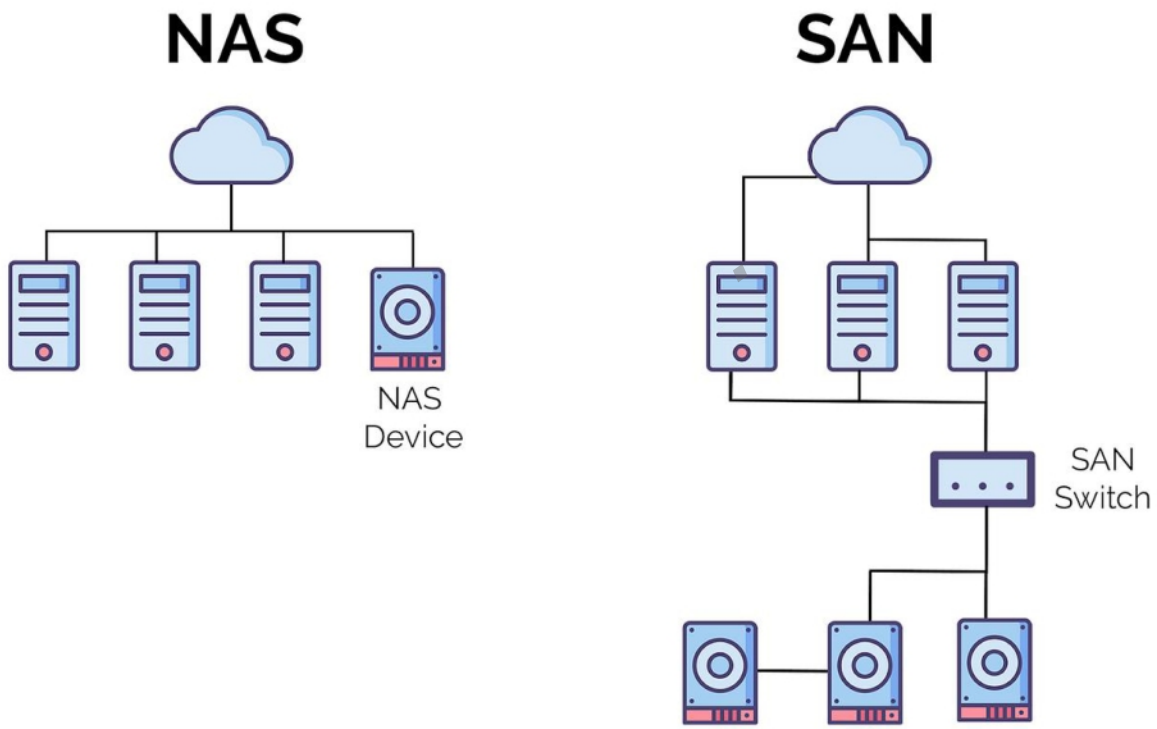
- Copy-on-Write (COW): Ensures data integrity by never overwriting existing data directly.
- Snapshots & Clones: Enables efficient backup and data recovery.
- Built-in RAID Support: Eliminates the need for traditional RAID controllers.
- Self-Healing Data: Uses checksums to detect and correct corruption.
- Scalability: Supports massive amounts of data and storage pools.

Modern File System

ZFS (Zettabyte File System)

Use Cases

- Enterprise storage systems (NAS-Network Attached Storage, SAN-Storage Area Network)
- Data centers and cloud computing



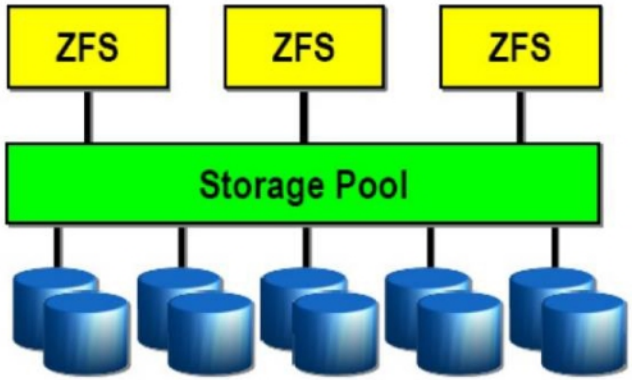
Modern File System

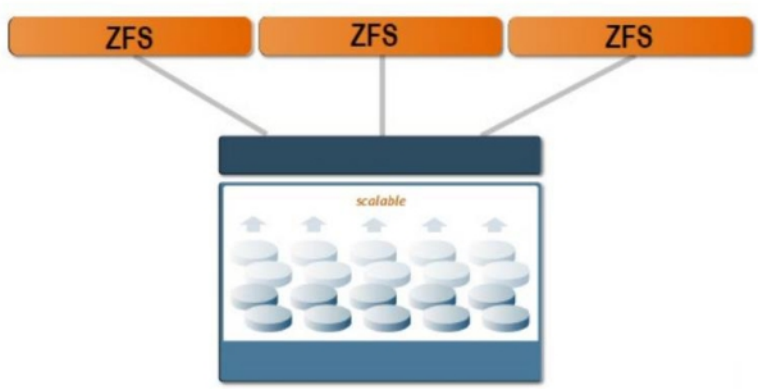
ZFS Design Principle

- ZFS was built from the ground up to address modern storage challenges rather than relying on legacy file system architectures.

1. Pooled Storage

- Traditional storage uses fixed volumes, which can be restrictive. ZFS eliminates the concept of rigid volumes and introduces **storage pools**, making it more flexible. Similar to how virtual memory abstracts physical RAM, ZFS dynamically allocates storage resources.





- ZFS file systems are built on top of virtual storage pools called Zpools. A Zpool is constructed of devices, real or logical. They are constructed by combining block devices using either mirroring or RAID-Z.

Modern File System

ZFS Design Principle

- ZFS was built from the ground up to address modern storage challenges rather than relying on legacy file system architectures.

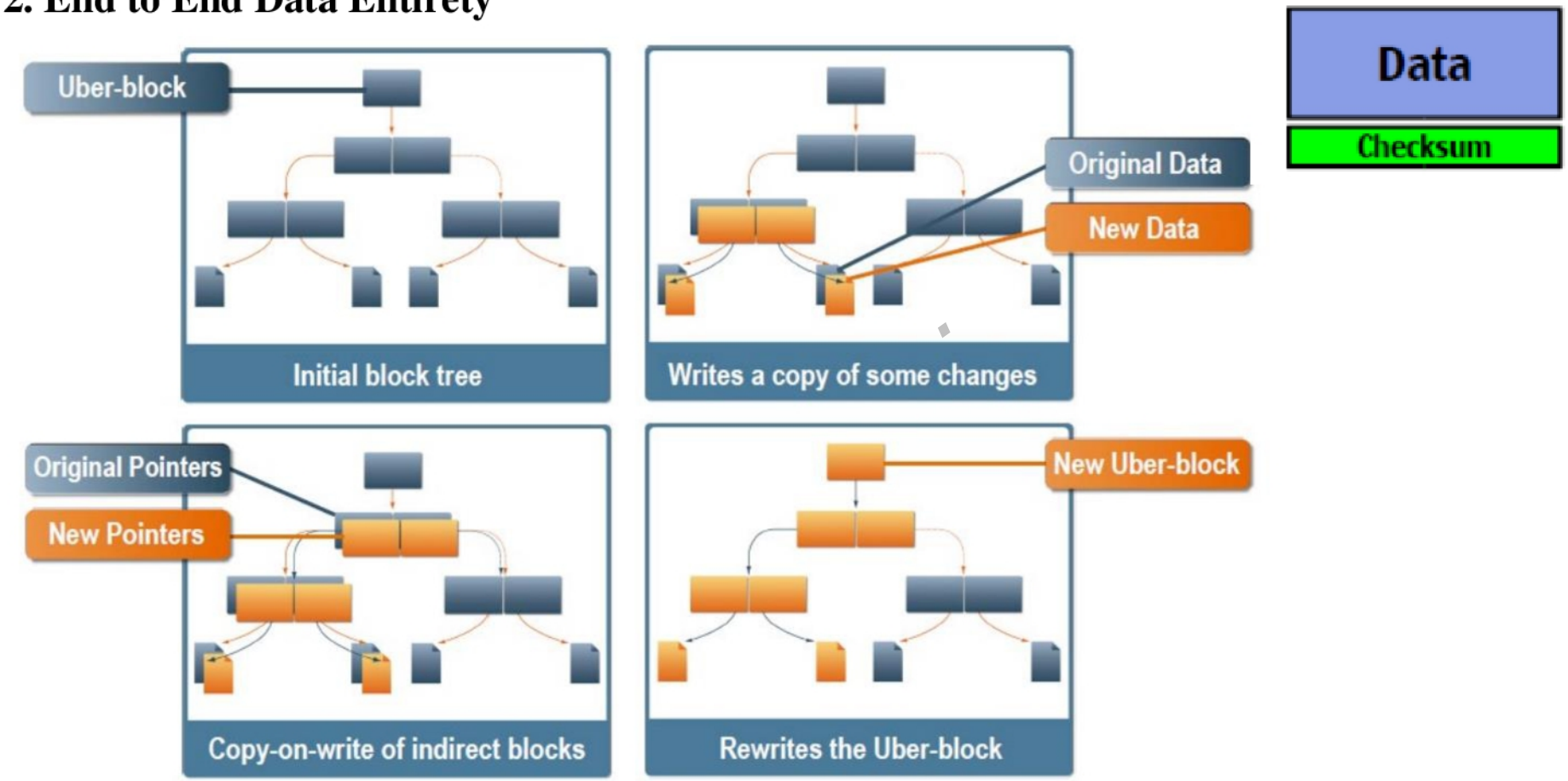
2. End to End Data Entirety

- Older file systems often neglected comprehensive data integrity checks due to high costs. ZFS data integrity model includes
    1. **Everything is Copy-on-Write (COW)** i.e. no overwrite on live data.
    2. Everything is **transactional** i.e. related changes succeed or fail as a whole.
    3. Everything is Checksumed i.e. ZFS ensures **end-to-end data integrity**, protecting data from silent corruption through checksums and self-healing mechanisms.
    4. Enhanced data protection via **mirrored pool , RAID-Z and disk scrubbing** (uses checksum to verify the integrity of all data)

Modern File System

ZFS Design Principle

2. End to End Data Entirety

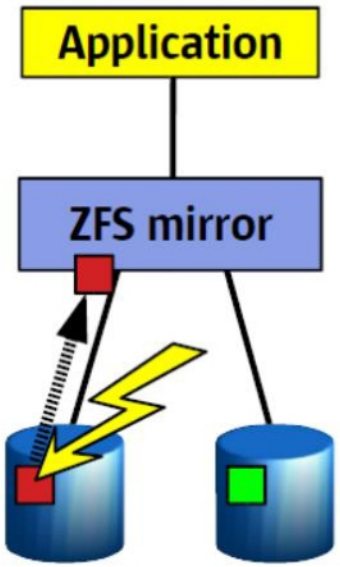


Modern File System

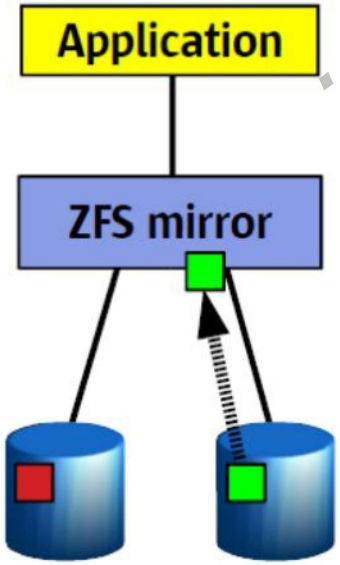
ZFS Design Principle

Self-Healing data in ZFS

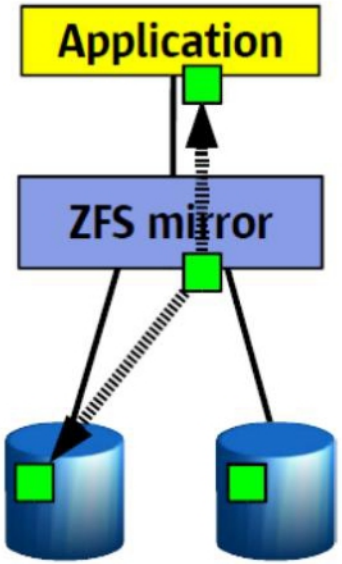
1. Application issues a read. ZFS mirror tries the first disk. Checksum reveals that the block is corrupt on disk.



2. ZFS tries the second disk. Checksum indicates that the block is good.



3. ZFS returns good data to the application and repairs the damaged block.



Modern File System

ZFS Design Principle

Traditional RAID-4 and RAID-5 vs RAID-Z

- **Parity Updates Require Read-Modify-Write (Slow Process):** When new data is written, RAID-5 does not simply overwrite the old parity but must read the old data and old parity, then recalculate the new parity before writing it. This introduces two additional disk reads before writing, making partial stripe writes slow.
- **Write Hole Issue (Data Corruption Risk):** If power is lost between data and parity writes, the RAID array can become inconsistent, leading to data corruption. Workarounds exist, like using non-volatile RAM (NVRAM) to ensure parity writes complete, but this is expensive.
- **Cannot Detect or Correct Silent Data Corruption:** Traditional RAID-5 does not include checksumming to verify data integrity. If a disk develops bad sectors or bit flips occur, corrupted data may be returned without detection.

- RAID-Z, part of ZFS, eliminates RAID-5 flaws by using copy-on-write (COW) and checksums, ensuring atomic writes to prevent the write hole issue, avoiding the read-modify-write penalty for faster performance, enabling self-healing with automatic data correction, and offering better scalability with dynamic parity distribution for improved reliability and efficiency.

Modern File System

ZFS Design Principle

3. Transaction Operation

- Ensures that data modifications maintain a consistent on-disk format, preventing corruption.
- Transactions are reordered and optimized for performance, providing significant gains over traditional file systems.

Modern File System

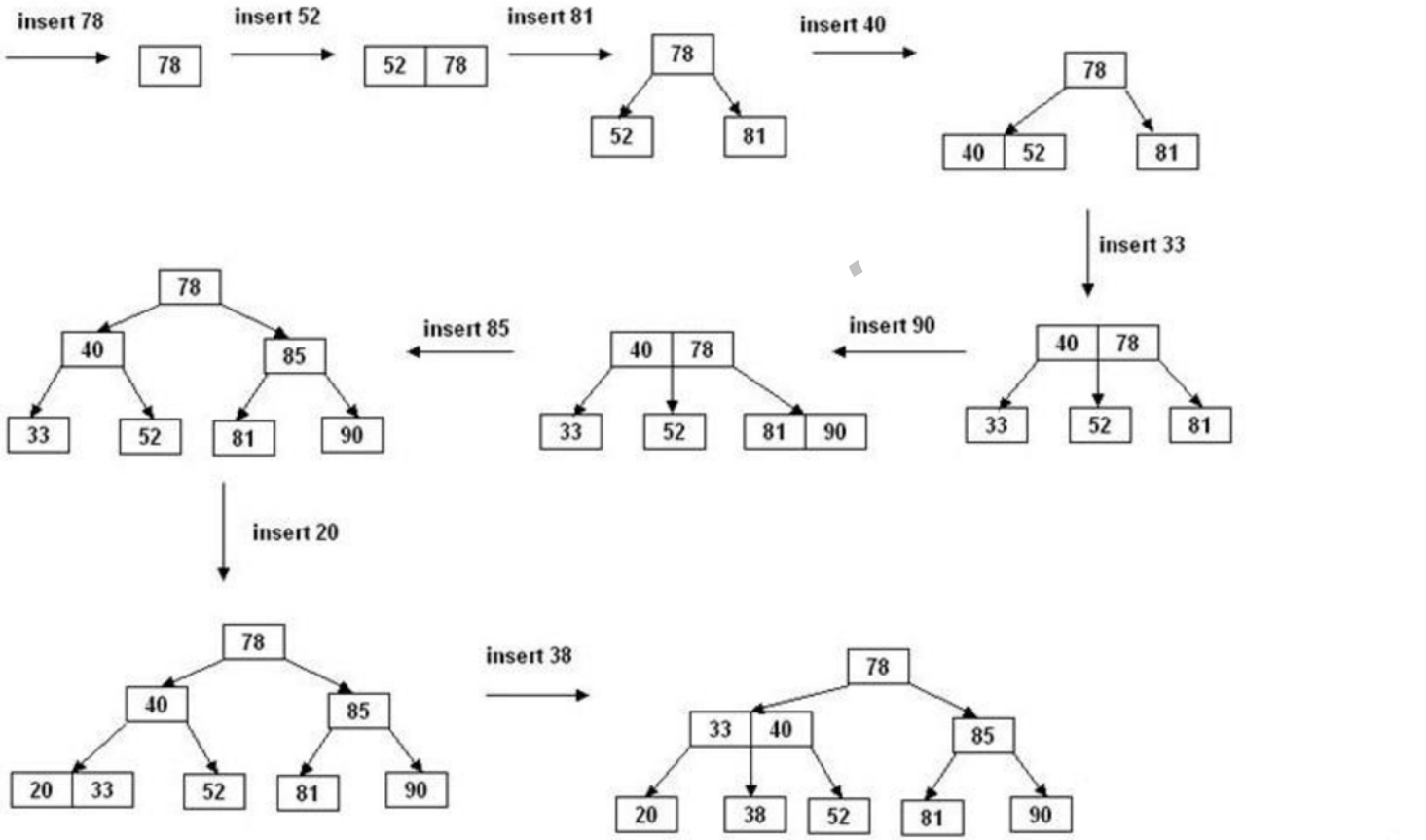
Modern File System: Btrfs

- **Btrfs (pronounced B-tree FS) is a modern copy-on-write (COW) file system developed by Oracle in 2007 and later adopted into the Linux kernel.**
- It was designed to address the limitations of traditional file systems like ext that addresses the lack of Snapshot and checksum and provide advanced storage features, better scalability, and improved reliability.
- **Btrfs is primarily used in Linux-based environments, including enterprise servers, NAS systems, and cloud storage platforms.**
- The whole idea of Btrfs is build around B-Trees and the implementation of COW on B-trees. Actually, all structures present in a Btrfs are stored using B-trees (like inodes, files, directory entries, block pointers) making operations like searching, insertion, and deletion highly efficient.
- A B-Tree (Balanced Tree) is a self-balancing tree structure commonly used in databases and file systems for efficient data retrieval. Btrfs uses B-Trees extensively for organizing metadata and data structures.

Modern File System

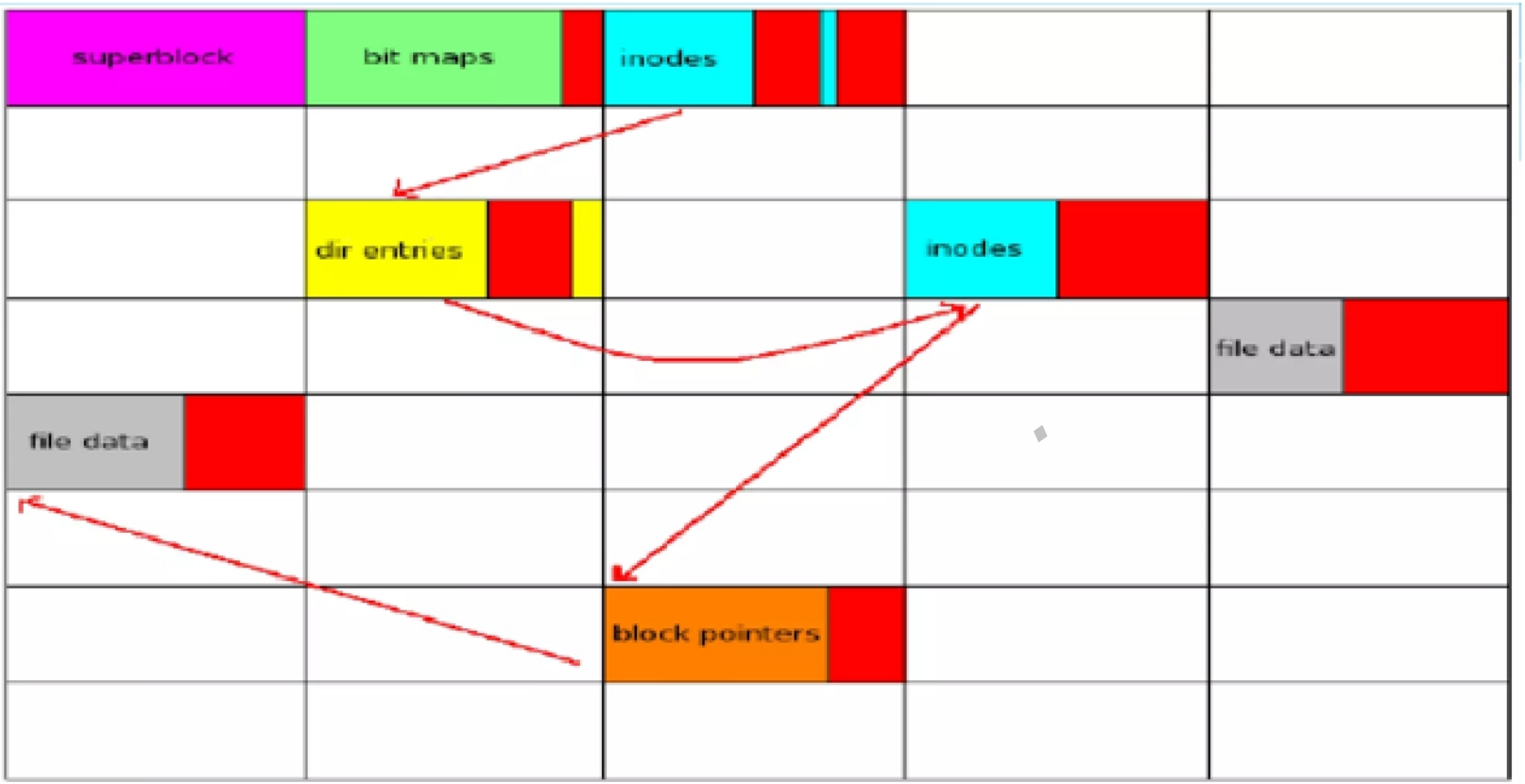
Modern File System: Btrfs

- Example: Insert the keys 78, 52, 81, 40, 33, 90, 85, 20, and 38 in this order in an initially empty B-tree of order 3



Modern File System

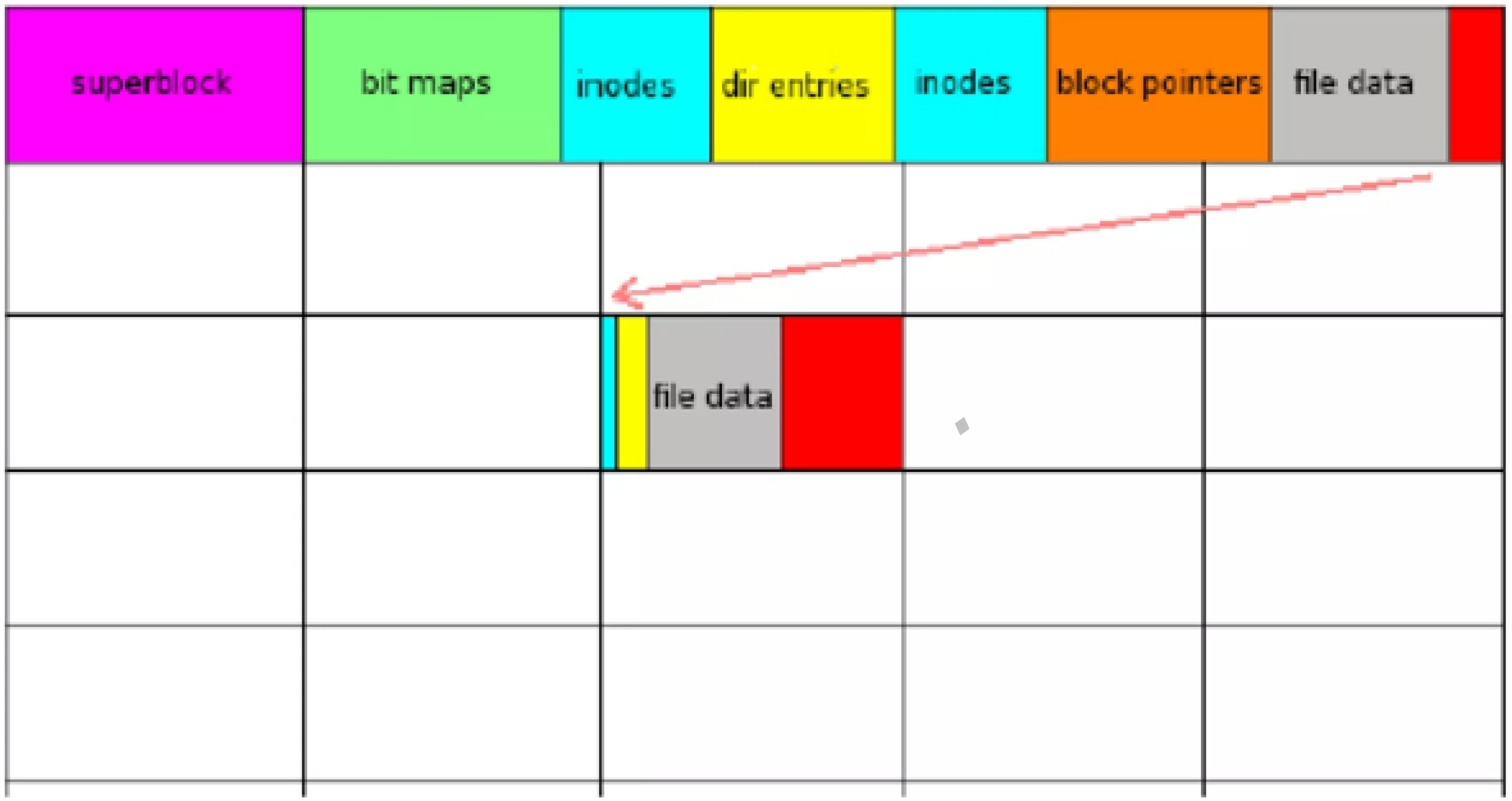
Modern File System: Btrfs



Fig; Data Organization in Old File System

Modern File System

Modern File System: Btrfs



Fig; Data Organization in Brtfs

Modern File System

Modern File System: Btrfs

Key components in above diagram includes

- Superblock: Stores metadata about the file system. Meta data includes file system type(ext, zfs, Btrfs etc.), size of file system, block size, number of Inodes and blocks etc.
- Bitmaps: Track free and allocated blocks.
- Inodes: Store file metadata (permissions, ownership, size, and pointers to data blocks).
- Directory entries: Map file names to inode numbers.
- Block pointers: Store pointers to data blocks.
- File data: The actual content of files.

- In old file system, the Inodes, block pointers, directory entries, and file data are stored in different places. Multiple arrows indicate several steps needed to locate and retrieve data. Scattered storage leads to more disk seeks and potential fragmentation.
- But in Btrfs, the metadata and small file data are stored in the same block, reducing seeks. Uses extents instead of individual block pointers, improving efficiency. Fewer arrows, indicating faster data retrieval and less fragmentation.

Modern File System

Modern File System: Btrfs

- Traditional file systems manage files using fixed-size blocks (e.g., 4KB per block). If a file grows beyond one block, multiple blocks are allocated, often in non-contiguous locations, leading to fragmentation.
- Btrfs optimizes storage using B-Trees and variable size extents(contiguous chunks), improving performance and reducing fragmentation.

Modern File System

Modern File System: Btrfs

Key Features of Btrfs

- **Copy-on-Write (COW):** Unlike traditional file systems that overwrite existing data, Btrfs writes new data to a different location before updating metadata. This ensures data integrity because the original data remains unchanged until the new write is confirmed which helps prevent data corruption during power failures or crashes.

- **Built-in RAID Support:** Btrfs has native support for RAID-0, RAID-1, RAID-10, RAID-5, and RAID-6, eliminating the need for separate RAID management software. Unlike traditional RAID solutions, Btrfs handles RAID at the file system level, allowing flexible disk management and automatic error correction.

Modern File System

Modern File System: Btrfs

Key Features of Btrfs

- **Snapshots (Point-in-Time Copies):** Btrfs allows instant, lightweight snapshots, capturing the state of the file system at a specific point. Snapshots use COW, meaning only changed data is written, saving space. Useful for backup, system rollback, and versioning.

- **Data Integrity with Checksums:** Every data block and metadata block in Btrfs has a CRC checksum. Automatically detects and repairs silent data corruption. If corruption is detected, Btrfs can recover data using a RAID redundancy mechanism

Modern File System

Key Features of Btrfs

- **Transparent Compression:** Btrfs supports inline compression using:
  - Zlib (Slow but High Compression Ratio)
  - LZO (Faster but lower compression)
  - ZSTD (Balanced performance and compression)

  Improves storage efficiency and reduces SSD wear by minimizing write amplification.

- **Online Resizing and Dynamic Storage Management:** Btrfs volumes can be resized online (increased or decreased) without unmounting. Supports adding and removing disks dynamically, making it a flexible choice for storage expansion.

Modern File System

Key Features of Btrfs

- **Self-Healing and Auto-Repair:** If corruption is detected via checksum verification, Btrfs can automatically repair the data using redundant copies (if RAID-1, RAID-5 is enabled). Helps prevent silent data corruption that traditional file systems cannot detect.

- **Send/Receive (Efficient Data Transfer):** Btrfs can efficiently transfer file system **snapshots** between different systems. Used for incremental backups, remote replication, and disaster recovery.

Distributed File System

Introduction to Google file system

- Traditional filesystems organize and track files on a single storage device, providing a logical structure of directories and files that users and applications can interact with.
- Example:
  - FAT32 and NTFS for Windows
  - ext4 and XFS for Linux
  - APFS for macOS
- As computing needs grew beyond single machines, distributed file systems (DFS) emerged to allow multiple clients to access shared storage resources across a network. **A DFS manages files and storage across multiple servers or nodes, while presenting a unified view to clients.**

Key Characteristics of Distributed File Systems:

- Scalability: Data may spread many storage nodes
- Location Transparency: Clients don't need to know physical file locations
- Reliability: Fault tolerance through data replication
- Performance: Concurrent access from multiple clients

Distributed File System

GFS

- GFS was designed to meet Google's unique requirements for storing and processing massive amounts of data across thousands of commodity machines.to Google file system

Assumptions

- ✓ COTS (Commercial Off-The-Shelf)) x86 servers
- ✓ Few million files, typically 100MB or larger
- ✓ Large streaming reads, small random reads
- ✓ Sequential writes, append data
- ✓ Multiple concurrent clients that append to the same file
- ✓ High sustained bandwidth more important than low latency

Distributed File System

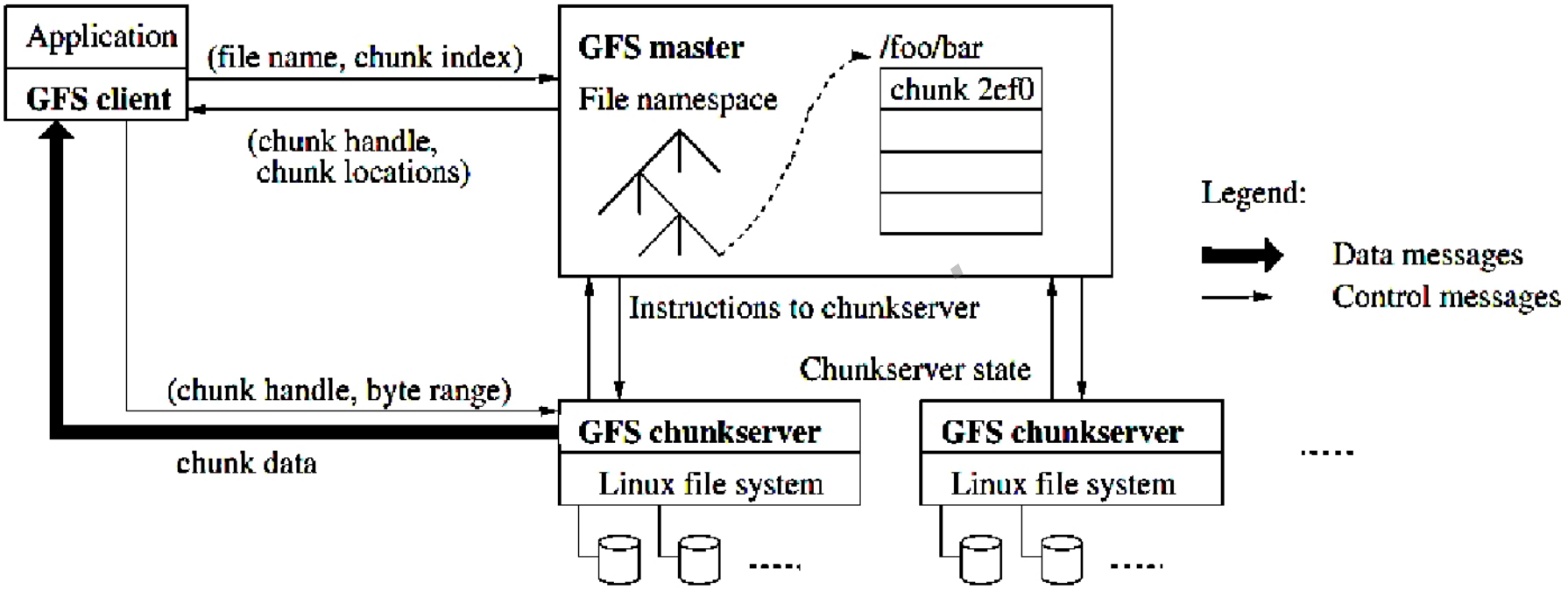
GFS: Interface

Familiar file system interface API.

- Usual files operations:
  - create
  - delete
  - open
  - close
  - read
  - write
- Enhanced files operations:
  - snapshots
  - record append : concurrent atomic append from multiple clients supported

Distributed File System

GFS: Architecture



Distributed File System

GFS: Architecture

- A GFS cluster consists of **a single master, multiple chunk servers, and multiple clients.**
- Each component runs on a commodity Linux machine as a user-level server process.
- Files are divided into **fixed-size chunks, each identified by a unique 64-bit chunk handle**.
- Chunk servers store chunks as Linux files and replicate them for reliability (default: three replicas).
- The master manages **metadata, chunk leases, garbage collection, and chunk migration.**
- **Clients communicate with the master for metadata but interact directly with chunk servers for data**.
- Neither clients nor chunk servers cache file data, relying on Linux's buffer cache instead.

Distributed File System

GFS: The Master Node

- Since the master is single, and has global knowledge of the whole system, this drastically simplifies the design
- Responsible for all system-wide activities
  - ✓ managing chunk leases, reclaiming storage space, load-balancing
- Maintains all file system metadata
  - ✓ Namespaces, ACLs, mappings from files to chunks, and current locations of chunks
  - ✓ all kept in memory, namespaces and file-to-chunk mappings are also stored persistently in operation log
- Periodically communicates with each chunk server in Heartbeat messages
  - ✓ This let's master determines chunk locations and assesses state of the overall system
  - ✓ The chunk server has the final word over what chunks it does or does not have on its own disks – not the master

Distributed File System

GFS: The Master Node

- For the namespace metadata, master does not use any per-directory data structures – no Inodes. In a traditional file system, each directory (/user, /user/docs, etc.) would have its own Inode structure linking files under it. However in GFS, Every file and directory is represented as a node in a simple lookup table as below, mapping pathnames to metadata (e.g., chunk references, permissions).

Pathname	Metadata (Chunks, ACLs, etc.)
`/user/docs/report1.txt`	Chunk 1, Chunk 2, ...
`/user/docs/report2.txt`	Chunk 3, Chunk 4, ...
`/user/videos/movie.mp4`	Chunk 5, Chunk 6, ...

Distributed File System

GFS: The Master Node

- Stored efficiently using prefix compression (< 64 bytes per namespace entry). Since GFS stores full pathnames, prefix compression helps reduce memory usage.
- For the above file paths, instead of storing full names redundantly, GFS applies prefix compression like this:

```
/user/docs/ → (Common Prefix)
/report1.txt → (Metadata for report1.txt)
/report2.txt → (Metadata for report2.txt)
/user/videos/ → (Common Prefix)
/movie.mp4 → (Metadata for movie.mp4)
```

- Further, each node in the namespace tree has a corresponding read-write lock to manage concurrency. Multiple reads can happen simultaneously (e.g., multiple clients listing files in /user/docs). Writes (modifications) require exclusive access to prevent conflicts (e.g., renaming /user/docs/report1.txt).
- Because all metadata is stored in memory, the master can efficiently scan the entire state of the system periodically in the background

Distributed File System

GFS: The Master Node

- But the master is (hopefully) never the bottleneck
  - Clients never read and write file data through the master; client only requests from master which chunk servers to talk to
  - Master can also provide additional information about subsequent chunks to further reduce latency
  - Further reads of the same chunk don't involve the master, either
- Master state is also replicated for reliability on multiple machines, using the operation log and checkpoints

Distributed File System

GFS: Chunk and Chunk Server

- Files are divided into fixed-size chunks, which has an immutable, globally unique 64-bit chunk handle(unique 64bit code that helps in tracking and locating chunks efficiently).
- Consider a file /user/videos/movie.mp4 which is 200 MB in size. It will be divided into 3 chunks of 64 MB each and one chunk of 8 MB:

Chunk Handle	File Offset	Size
0xABC12345	0MB – 64MB	64 MB
0xDEF67890	64MB – 128MB	64 MB
0x1234ABCD	128MB – 192MB	64 MB
0x5678EFGH	192MB – 200MB	8 MB

Distributed File System

GFS: Chunk and Chunk Server

- By default, each chunk is replicated three times across multiple chunk servers (user can modify amount of replication) . This is done for fault tolerance. Assume the above Chunk 0xABC12345 (first chunk of movie.mp4) is replicated on three different chunk servers:

Chunk Handle	Chunk Server 1	Chunk Server 2	Chunk Server 3
0xABC12345	Server A	Server B	Server C

- If Chunk Server A fails, GFS will automatically create another replica from an existing copy to maintain the required replication level.
- Chunk servers store the chunks on local disks as Linux files

Distributed File System

GFS: Chunk and Chunk Server

Metadata per chunk is < 64 bytes (stored in master)

- Current replica locations
  - ✓ which chunk server has copies for given chunk handle. For chunk 0xABC12345 the metadata in the master may look like
  - ✓ { "chunk_handle": "0xABC12345", "replica_locations": ["Server A", "Server B", "Server C"] }
- Reference count (useful for copy-on-write)
  - ✓ The reference count tracks how many files or processes are referencing a chunk. This is crucial for Copy-on-Write (CoW) operations.
  - ✓ If a user makes a copy of a large file, GFS does not immediately duplicate the chunks. Instead, it increases the reference count by one and creates a new file entry pointing to the same chunk. However, If the copied file is modified, GFS creates a new chunk instead of modifying the original, ensuring data integrity.
- Version number (for detecting stale replicas(outdated or invalid replica of a chunk that no longer matches the latest version of the data)
  - ✓ Each chunk has a version number to detect stale copies. When a chunk is modified, its version number is incremented.

Distributed File System

GFS: Chunk and Chunk Server

- GFS uses 64MB chunks, which is much larger than traditional file systems (which typically use 4KB or 8KB blocks). This design choice has both advantages and disadvantages

Disadvantages:

✓ Wasted space due to internal fragmentation. A 10MB file still occupies one full 64MB chunk, wasting 54MB of space

✓ Small files consist of a few chunks, which then get lots of traffic from concurrent clients. A 10MB file fits in one chunk, so all clients reading the file will request the same chunk server, causing high traffic and delays. This can be mitigated by increasing the replication factor i.e. store same data in multiple server

Distributed File System

GFS: Chunk and Chunk Server

Advantages:

- Reduces clients' need to interact with master. Since, The GFS master stores metadata (e.g., file-to-chunk mapping, chunk locations). If chunks were smaller (e.g., 4KB), clients would frequently contact the master, causing a bottleneck.

✓ Since client is likely to perform many operations on a given chunk, keeping a persistent TCP connection to the chunk server reduces network overhead.

✓ Reduces the size of the metadata stored in master → metadata can be entirely kept in memory

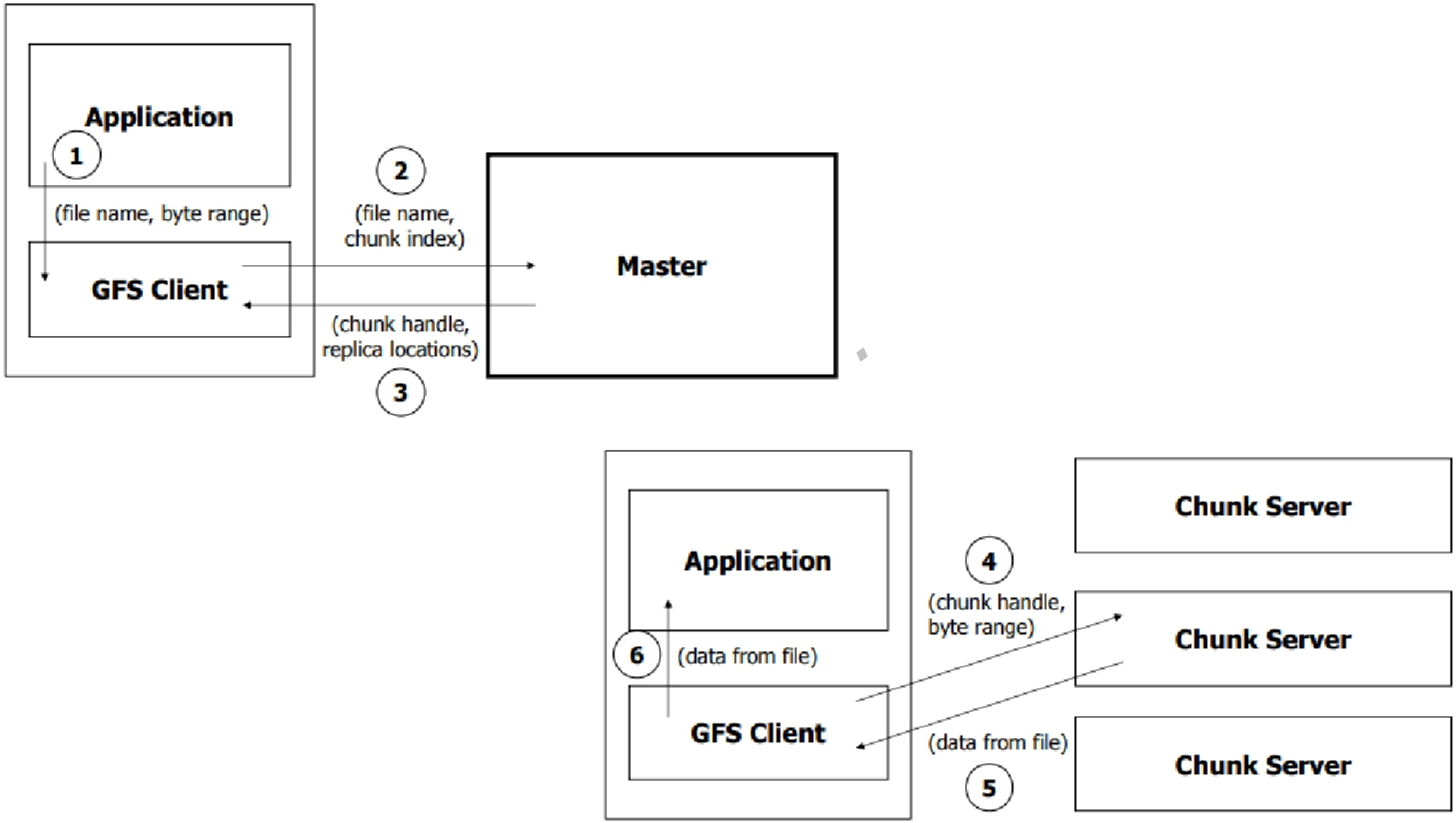
Distributed File System

GFS: Relaxed Consistency Model

- GFS uses a relaxed consistency model to manage how data is accessed and modified across its distributed file system.
- Unlike traditional file systems that enforce strict consistency, GFS aims to prioritize scalability, availability, and performance, especially in the context of large-scale data processing.
- Achieving strong consistency (where all nodes see the same data immediately) can reduce performance and availability, especially in large distributed systems.
- Many DFS including GFS adopt **eventual consistency models**, which allow temporary inconsistencies to exist but ensure they resolve over time. GFS i.e. changes made to a file might not be immediately visible to all clients. Over time, all replicas of the file will converge to the same state, but this doesn't guarantee immediate consistency after a write.

Distributed File System

GFS: Read Operation



Distributed File System

GFS: Read Operation

- The application sends a read request specifying:
  Filename (e.g., /data/logs/server.log)
  Byte range (e.g., from byte 1000 to 5000)
- The GFS client (running on the application's machine) translates the request from:
  (filename, byte range) → (filename, chunk index)
- It calculates which chunk index contains the requested byte range.
  Example:
    If a chunk size is 64MB, then byte 1000 belongs to chunk 0.
    If a request includes byte 70MB, it will involve chunk 1 as well.
- Master Responds with Chunk Handle(a unique identifier for the chunk) & Replica Locations(list of chunk servers storing this chunk). The GFS master stores metadata about file-to-chunk mappings.
- The client chooses one of the replica locations (usually the nearest or least loaded). The client then sends a request directly to the chunk server, including: Chunk handle, Byte range
- The selected chunk server reads the requested byte range from its local storage. It sends the data back to the client.
- The GFS client receives the data from the chunkserver. GFS Client Forwards Data to the Application. If the requested byte range spans multiple chunks, steps 4–6 are repeated for each chunk.

Distributed File System

GFS: Example Use Case

- Imagine a video streaming platform that stores large video files in GFS.
- A 2GB video is split into multiple 64MB chunks.
- The GFS master assigns these chunks to different chunk servers, ensuring that each chunk has at least three replicas.
- A user starts streaming the video, and the client queries the master for chunk locations.
- The client retrieves video chunks directly from chunk servers without overloading the master.
- If a chunk server storing a video chunk fails, the master detects the issue through missed heartbeats and assigns a replica to replace it, ensuring uninterrupted playback.

Distributed File System

GFS: Record Write Operation

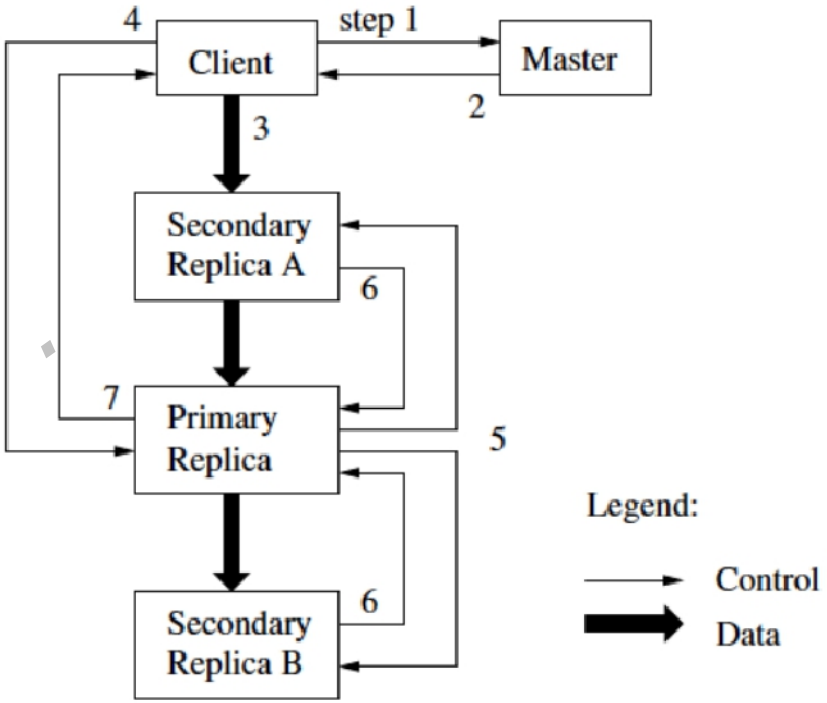
- To ensure consistency and correctness in mutations (writes or Record appends) across multiple chunk replicas, Google File System (GFS) follows **a lease-based mechanism** for managing mutation order.
- In this mechanism, **master finds the chunk servers that have the chunk and grants a chunk lease to one of them**. This server is called the primary, the other servers are called secondaries
- Master grants a chunk lease for 60 seconds to one of the replicas: the primary replica (primary can request lease extension: via heartbeat message).
- The primary replica picks a serial order for all mutations to the chunk . All replicas follow this order when applying mutations (global mutations order defined first by the lease grant order chosen by the master, and within a lease by the serial numbers assigned by the primary replica)

Distributed File System

GFS: Record Write Operation

When any Application originates write request.

1. GFS client translates request from (filename, byte_range) -> (filename, chunk index), and sends it to master.
2. Master responds with chunk handle and (primary + secondary) replica locations.
3. Client pushes write data to all locations. Data is stored in chunk servers' internal buffers.
4. Client sends write command to primary.
5. Primary determines serial order for data instances stored in its buffer and writes the instances in that order to the chunk. Then, Primary sends serial order to the secondaries and tells them to perform the write.
6. Secondaries respond to the primary acknowledging that they have completed their operations.
7. Primary responds back to client. Any errors encountered at the replaces are reported to the client.
8. Note: If write fails at one of chunk servers, client is informed and retries the write.



Distributed File System

GFS: Record Append Operation

- The record append operation in Google File System (GFS) is a special type of write operation designed to allow clients to efficiently append data to a file in a sequential, append-only fashion.
- This operation is particularly useful in scenarios where atomicity is important, and where data needs to be added to the end of the file without overwriting any existing data such as in log files, event streams, or other use cases that demand data to be appended at the end without overwriting previous data.
- The key idea is that the client does not specify where in the file the data should go, but instead just specifies the data to append. GFS automatically figures out where to append the data within the file and manages the file offsets for the client.

Distributed File System

GFS: Record Append Operation

- Common use cases for record append include:
  - Log files: Logs are usually written sequentially as new entries are added.
  - Event streams: Data generated by multiple sources that must be added to the end of a file without altering existing data.
  - Merging results from multiple machines or processes, such as when results from different MapReduce tasks are aggregated into a single file.

Distributed File System

GFS: Record Append Operation

Steps In Record Append

- Application originates record append request. Maximum size of an append record is 16 MB
- The client initiates a record append request by specifying the data to be appended. The client does not specify the offset or position in the file where the data should go. GFS client translates request and sends it to master.
- The client sends the append request to the master server.
- Master responds with chunk handle and (primary + secondary) replica locations.
- Client pushes write data to all locations.

Distributed File System

GFS: Record Append Operation

- The primary chunkserver is responsible for ensuring that the append fits in the chunk. GFS divides files into fixed-size chunks (typically 64 MB). The primary chunkserver checks whether the data being appended can fit within the current chunk.
  - ❑ If the data fits:
    - ✓ The primary chunkserver appends the data to the chunk and tells the secondary replicas to do the same.
  - ❑ If the data doesn't fit:
    - ✓ The primary chunkserver will pad the chunk to the maximum size (64 MB), and then tell the secondary replicas to do the same.
    - ✓ The client will then be asked to retry and append the next part of the data to the next chunk. A new chunk will be created if necessary, and the client will be given the new chunk handle and the new replica locations.

Distributed File System

HDFS: Introduction

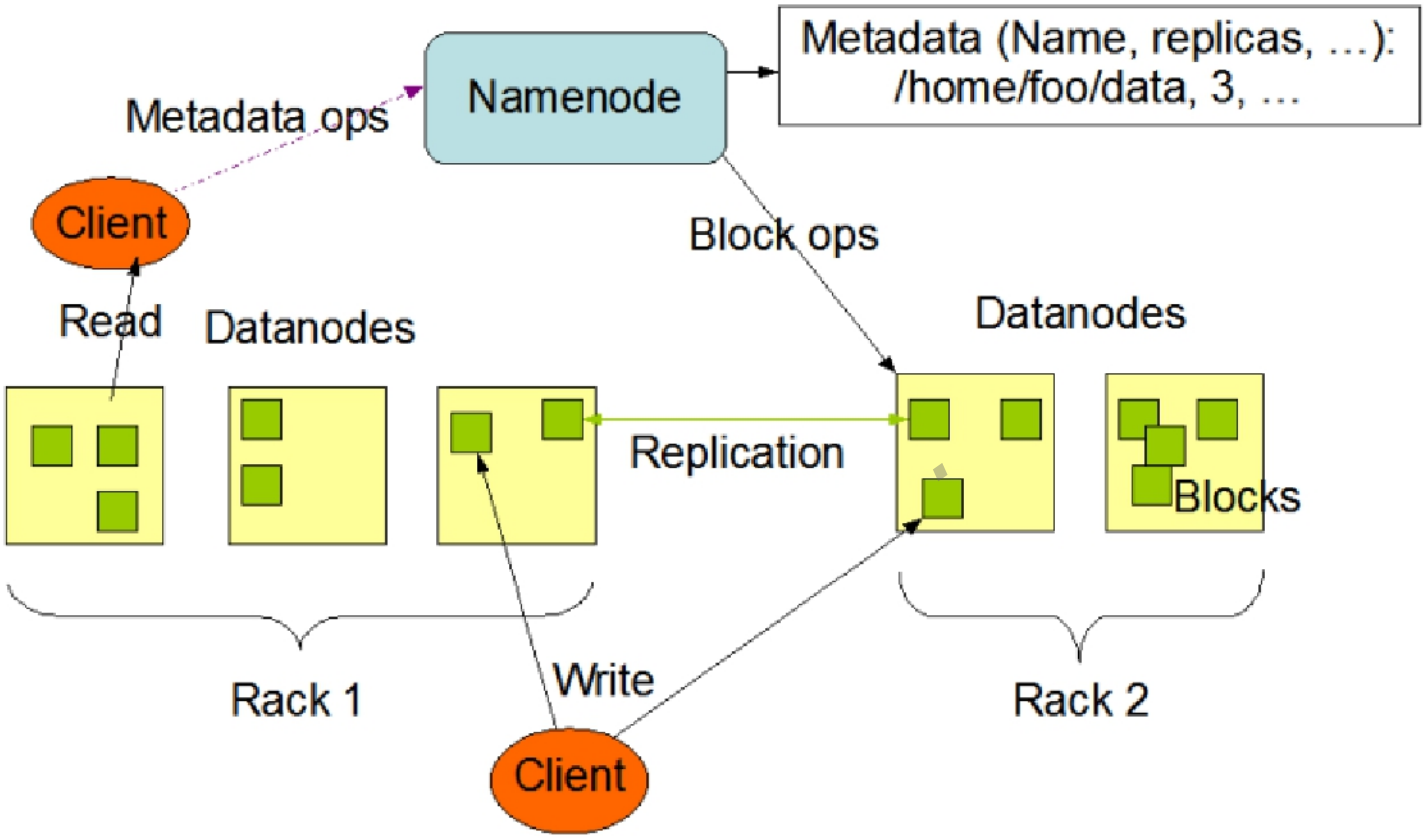
- **Hadoop is an open-source software framework designed for storing and processing large amounts of data in a distributed computing environment.**
- It was developed by the Apache Software Foundation and is based on the MapReduce programming model, which allows for the parallel processing of large datasets.
- Hadoop basically provides API & implementation for working with mapReduce, DFS optimized for huge amount of data, handling failure in computation and data nodes, job configuration and scheduling etc.
- **The Hadoop Distributed File System is key components of Hadoop designed to store and manage large volumes of data across multiple machines (commodity hardware) in a distributed manner.**
- It provides high-throughput access to data, making it suitable for applications that deal with large datasets, such as bigdata analytics, machine learning, and data warehousing .
- **It is built in Java, which allows it to be highly portable and run on any machine that supports the Java Runtime Environment (JRE), enabling a wide range of deployment options across different systems.**

Distributed File System

HDFS: Introduction

Who uses Hadoop?

Distributed File System



Distributed File System

HDFS: Architecture

- HDFS follows a master/slave architecture with a single NameNode (master) and multiple DataNodes (slaves).
- HDFS is designed to be highly scalable, reliable, and efficient, enabling the storage and processing of massive datasets. Its architecture consists of several key components:
    1. NameNode
    2. DataNode
    3. Secondary NameNode
    4. HDFS Client
    5. Block Structure

Distributed File System

HDFS: Architecture

NameNode

- The NameNode is responsible for managing the file system namespace, executing operations like opening, closing, and renaming files and directories.
- Additionally, the NameNode maps file blocks to DataNodes, maintaining the metadata and the overall structure of the file system.
- This metadata is stored in memory for fast access and persisted on disk for reliability.

Key Responsibilities:

- Maintaining the filesystem tree and metadata.
- Managing the mapping of file blocks to DataNodes.
- Ensuring data integrity and coordinating replication of data blocks.

Distributed File System

HDFS: Architecture

DataNode

- DataNodes are the worker nodes in HDFS, responsible for storing and retrieving actual data blocks as instructed by the NameNode.
- Each DataNode manages the storage attached to it and periodically reports the list of blocks it stores to the NameNode.

Key Responsibilities:

- Storing data blocks and serving read/write requests from clients.
- Performing block creation, deletion, and replication upon instruction from the NameNode.
- Periodically sending block reports and heartbeats to the NameNode to confirm its status.

Distributed File System

HDFS: Architecture

HDFS Client

- The HDFS client is the interface through which users and applications interact with the HDFS.
- It allows for file creation, deletion, reading, and writing operations.
- The client communicates with the NameNode to determine which DataNodes hold the blocks of a file and interacts directly with the DataNodes for actual data read/write operations.

Key Responsibilities:

- Facilitating interaction between the user/application and HDFS.
- Communicating with the NameNode for metadata and with DataNodes for data access.

Distributed File System

HDFS: Architecture

Block Structure

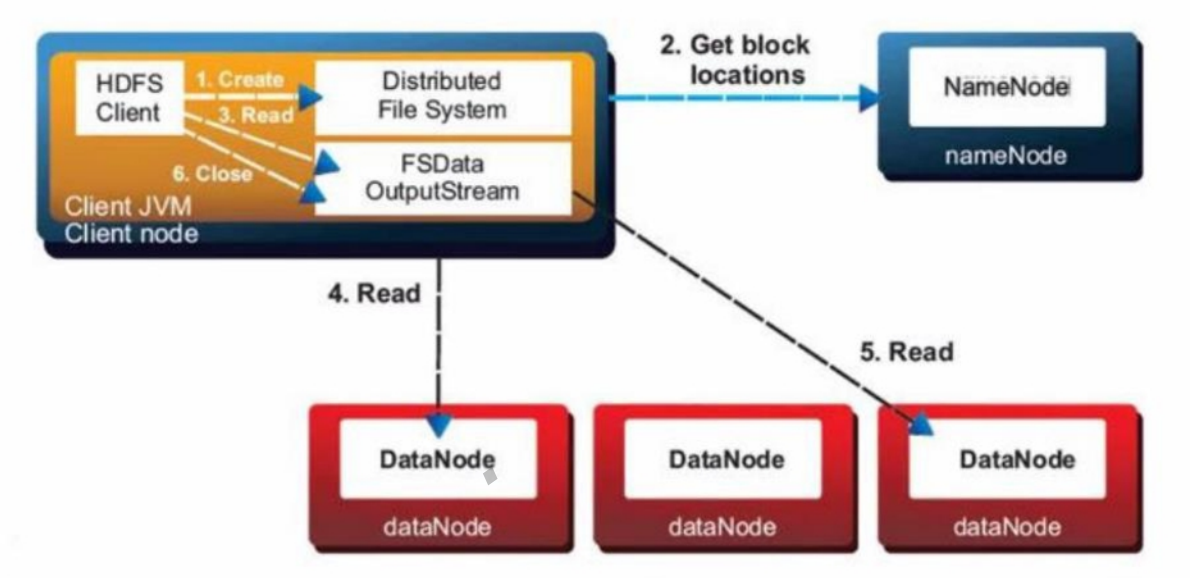
- HDFS stores files by dividing them into large blocks, typically 128MB or 256MB in size.
- Each block is stored independently across multiple DataNodes, allowing for parallel processing and fault tolerance.
- The NameNode keeps track of the block locations and their replicas.

Key Features:

- Large block size reduces the overhead of managing a large number of blocks.
- Blocks are replicated across multiple DataNodes to ensure data availability and fault tolerance.

Distributed File System

HDFS: Read



FSDataOutputStream is a class in the Hadoop FileSystem (HDFS) API that is used for writing data to files in the Hadoop Distributed FileSystem (HDFS). It is a subclass of Java's OutputStream and provides a way for Hadoop applications to write data to HDFS files.

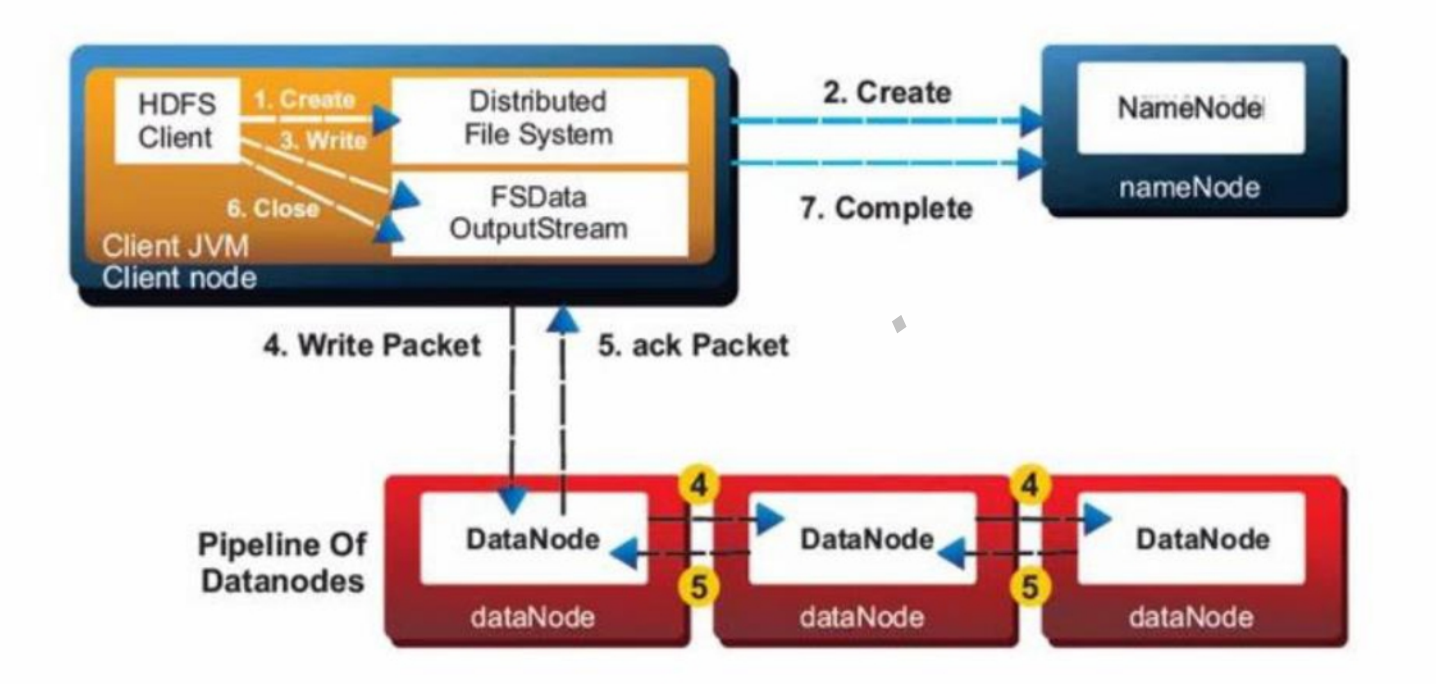
Distributed File System

HDFS: Read

- The HDFS client (inside a Java Virtual Machine - JVM) initiates a request to read a file stored in HDFS.
- The client contacts the NameNode, which maintains the metadata of the file system (file-to-block mapping, block locations, permissions, etc.). The NameNode responds with the locations of data blocks stored in DataNodes.
- After obtaining the block locations, the client decides which DataNode to contact based on proximity and network conditions.
- The client directly contacts the DataNode storing the required data block. The client reads the data in parallel from multiple DataNodes to increase performance.
- If the required data spans multiple blocks, the client sequentially requests data from other DataNodes where the next blocks are stored.
- Once all data is retrieved, the client closes the connection, and the read operation is complete.

Distributed File System

HDFS: Write



Distributed File System

HDFS: Write

- The HDFS client (inside a Java Virtual Machine - JVM) sends a request to create a file in HDFS.
- The NameNode (metadata server) checks if the file already exists and if the client has the necessary permissions. If valid, the NameNode allocates a set of DataNodes to store the file blocks.
- The client writes the data to the first DataNode in the pipeline.
- Data is Written to the Pipeline of DataNodes. HDFS follows a replication model to ensure fault tolerance.The first DataNode writes the data block and then forwards the data to the second DataNode. The second DataNode writes the block and forwards it to the third DataNode (if replication factor = 3).
- Once the last DataNode stores the block, it sends an acknowledgment packet (ACK) to the previous DataNode. This process continues in reverse order until the first DataNode sends an ACK back to the client.
- After writing all the blocks, the client sends a close request to the NameNode. Finally, The NameNode updates metadata and marks the file as successfully written.

Distributed File System

Ceph: Introduction

- Ceph is an open-source, software-defined storage system designed for high scalability, reliability, and performance.
- Ceph uniquely delivers object, block, and file storage in one unified system.It provides a unified storage solution that supports block storage (Block storage divides data into fixed-size chunks and stores them separately.) , object storage (Object storage stores data as objects along with metadata and a unique identifier.), and file storage(File storage which in one of the traditional organizes data in a hierarchical structure (files, directories).) under a single system thus making it a popular choice for cloud and large-scale storage infrastructures.
- Ceph is now maintained by the open-source community, primarily under the governance of the Ceph Foundation, which is part of the Linux Foundation.
- Used by serverlal cloud management platform like OpenStack, Kubernetes, and CloudStack to provide scalable storage.

Challenges in File and Storage

Scalability and Consistency Challenges in file and storage.

- With the exponential growth of data, modern storage systems face increasing challenges related to scalability (handling larger workloads efficiently) and consistency (ensuring reliable data operations across distributed environments).
- Scalability in file and storage systems refers to the ability to efficiently handle increased data volume, concurrent access, and geographical distribution. The major scalability challenges include:

1. **Metadata Bottlenecks**
    - Traditional file systems store metadata (file attributes, access permissions, Inode tables) in a centralized location, causing a single point of contention.
    - Problem: High metadata lookup times as the number of files grows (e.g., billions of small files in cloud storage).
    - Example: The inode table in Ext4 becomes a bottleneck at scale.
    - Solution: Distributed metadata architectures, which partition metadata across multiple nodes.

Challenges in File and Storage

Scalability and Consistency Challenges in file and storage.

2. Lock Contention

- High concurrent read/write access causes lock contention when multiple clients attempt to modify the same data.
- Problem: Traditional locking mechanisms (e.g., POSIX locks) introduce significant delays.
- Example: NFS suffers from lock contention when multiple clients write to the same file.
- Solution: Optimistic concurrency control (used in Btrfs and ZFS), and lock-free data structures in modern distributed storage.

3. High Latency in Large-scale Systems

- Large-scale storage spreads data across multiple nodes and regions, increasing access latency.
- Problem: Network latency dominates disk I/O in geo-distributed systems.
- Example: Amazon S3 has different performance characteristics in different AWS regions due to network latency.
- Solution: Data locality optimization (keeping frequently accessed data close to compute nodes) and multi-tier caching.

Storage Technologies

Scalability and Consistency Challenges in file and storage.

4. Replication Overhead

- Replicating data across nodes ensures availability but increases storage and network costs.
- Problem: Maintaining strong consistency across replicas requires synchronization, leading to delays.
- Example: HDFS replicates data 3 × by default, leading to storage inefficiency.
- Solution: Erasure coding (using Reed-Solomon encoding algorithm) used in Ceph and modern object stores reduces storage overhead while maintaining fault tolerance. Erasure coding works by splitting data into smaller chunks, adding redundancy through mathematical formulas, and distributing these chunks across different storage devices. This allows for efficient scalability because it reduces the overhead of replication while still offering fault tolerance. Here's a breakdown of the concept:

Storage Technologies

Scalability and Consistency Challenges in file and storage.

- Consistency ensures that data operations (reads/writes) produce predictable, correct results, even under failure conditions.
- However, achieving strong consistency in distributed storage introduces trade-offs.

1. CAP Theorem Trade-offs

- CAP Theorem: A distributed system can provide only two out of three properties:
  - Consistency (C): All nodes see the same data at the same time.
  - Availability (A): Every request receives a response despite some node fails.
  - Partition Tolerance (P): The system functions despite network failures despite network failure.
- Problem: Large-scale storage must sacrifice either strong consistency (C) or availability (A).
- Example: Amazon S3 prioritizes availability over strong consistency by using eventual consistency.
- In a real distributed system, network failures (P) are inevitable, so systems must choose between: CP → Prioritizing consistency over availability (strong consistency, but might delay responses). AP → Prioritizing availability over consistency (eventual consistency, but always responsive). Thus, only two out of three can be achieved at any given time.

Storage Technologies

Scalability and Consistency Challenges in file and storage.

1. CAP Theorem Trade-offs

Why Only Two at a Time?

Case 1: CP (Consistency + Partition Tolerance, but no Availability)

- The system remains consistent even in the presence of network partitions.
- But to maintain consistency, some requests might be blocked (i.e., no response), reducing availability.
- Example: Google Spanner, Zookeeper, HBase (prioritize strong consistency over availability).
- Scenario: A database replicates data across multiple nodes. If a network partition occurs, it will stop serving requests until nodes can synchronize, ensuring consistency but sacrificing availability.

Storage Technologies

Scalability and Consistency Challenges in file and storage.

1. CAP Theorem Trade-offs

Case 2: AP (Availability + Partition Tolerance, but no Consistency)

- The system is always available, even if network partitions occur.
- But some nodes might serve stale or inconsistent data due to a lack of synchronization.
- Example: DynamoDB, Cassandra, CouchDB (eventual consistency, prioritize availability).
- Scenario: If one node cannot communicate with others, it still serves data without checking for the latest updates, ensuring availability but sacrificing consistency.

Case 3: CA (Consistency + Availability, but no Partition Tolerance)

- A system that provides both consistency and availability must assume no network failures (i.e., no partitioning).
- But in real-world distributed systems, network failures are inevitable, making this case unrealistic.
- Example: Traditional relational databases (MySQL, PostgreSQL) in a single-node setup—they are CA only if there is no distribution.

Storage Technologies

2. Eventual vs. Strong Consistency

- Eventual Consistency: Updates are propagated asynchronously; some reads may see stale data. Used in NoSQL databases (Cassandra, DynamoDB), cloud storage (S3, GFS).
- Strong Consistency: Every read sees the latest write. Used in Zookeeper, Spanner, Ceph RBD but comes with performance penalties.
- Problem: Strong consistency requires global synchronization (e.g., distributed locking), increasing latency.
- Solution: Hybrid consistency model's example

3. Split-brain and Data Inconsistencies

- Occurs when network failures cause multiple partitions to act independently, leading to divergent data copies.
- Example: In a replicated storage system, two nodes may update the same data simultaneously, leading to conflicts.
- Solution: Quorum-based writes or leader-based consensus algorithms ensure that most nodes agree before committing updates.

Storage Technologies

4. Snapshot and Checkpoint Consistency

- File systems use snapshots for backups and recovery, but ensuring consistency across snapshots is complex.
- Problem: Snapshots taken at different times instances across nodes may lead to inconsistent recovery states.
- Example: In Btrfs, copy-on-write (COW) snapshots can have metadata inconsistencies if writes are interrupted (say due to crash).
- Solution: Journaling(available in ext3 and ext4) and write-ahead logs (available in Btrfs and ZFS) help maintain consistency across snapshots.

Storage Technologies

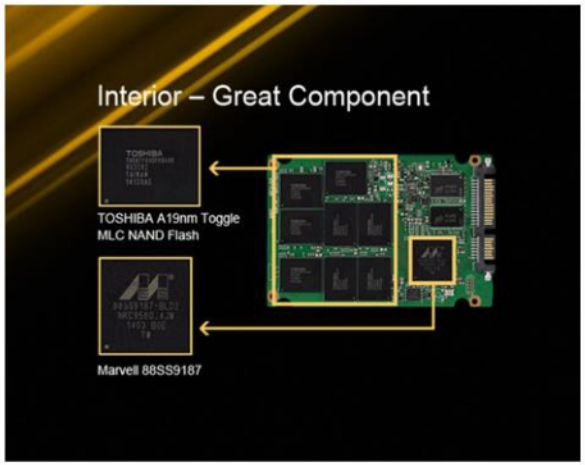
Introduction

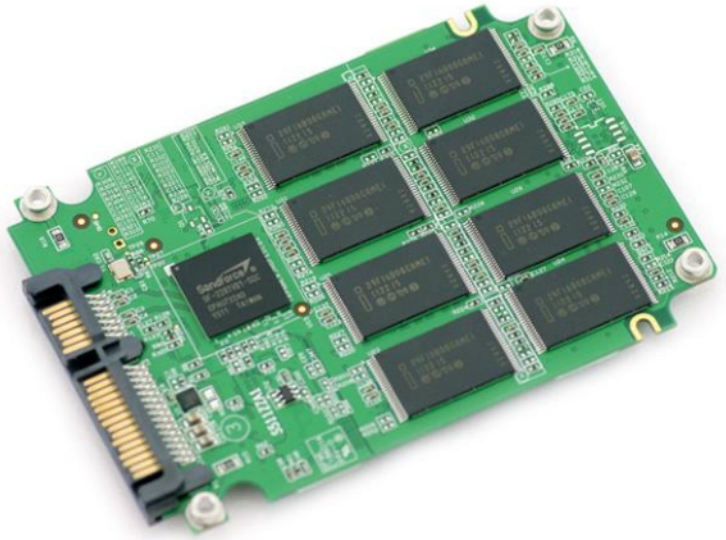
- In recent years, storage technology has evolved to meet the increasing demands for speed, capacity, and reliability.
- Three key technologies that play a crucial role in modern data storage solutions are Solid State Drives (SSDs), Non-Volatile Memory (NVM), and Persistent Memory (PMEM).

Storage Technologies

SSDs-(Solid State Disk

- SSDs are storage devices that use flash memory(NAND Flash) to store data, which is much faster than traditional Hard Disk Drives (HDDs) that rely on spinning disks.
- Flash memory is a type of non-volatile memory, meaning that it retains data even when power is lost that uses integrated circuit assemblies as memory to store data persistently.
- There are no moving mechanical components in SSD. This makes them different from conventional electromechanical drives such as Hard Disk Drives (HDDs) which contain movable read/write heads and spinning disks. SSDs are typically more resistant to physical shock, run silently, and have quicker access time, and lower latency compared to electromechanical devices.

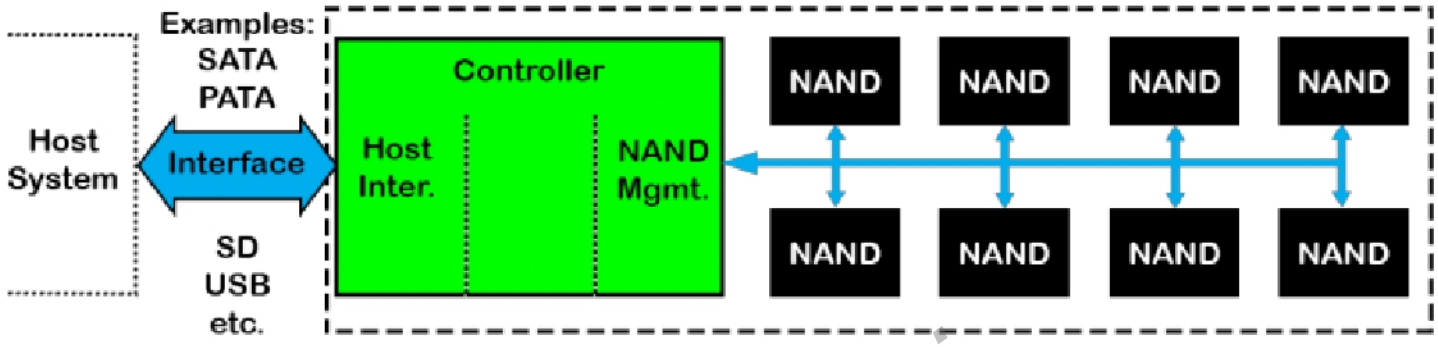




Storage Technologies

SSDs-Architecture

•silently, and have quicker access time, and lower latency compared to electromechanical devices.



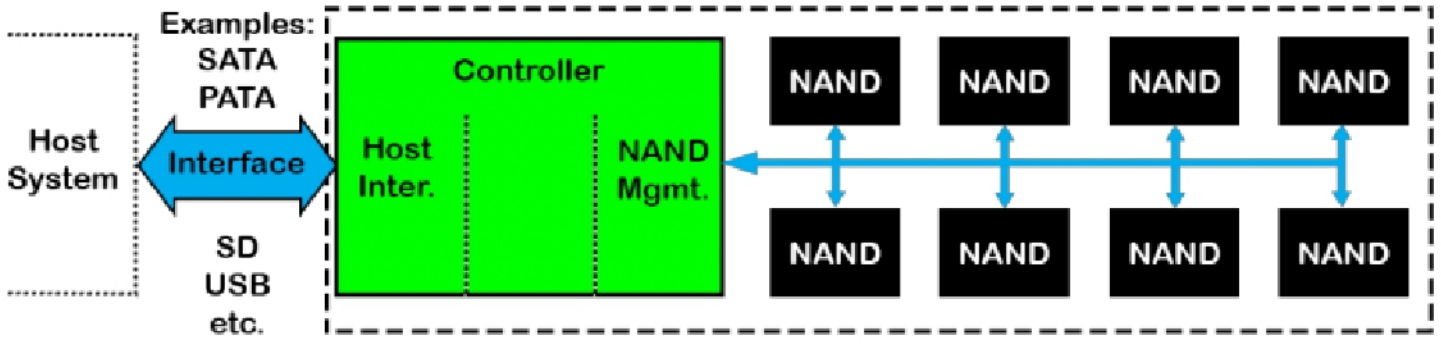
Basic Solid State Drive (SSD) Architecture

Main component are :

- **Interface:** The interface manages data transfer between the host (CPU) and the NAND flash controller. The Interface connects the NAND flash memory system to the external device (e.g., computer, microcontroller). It follows communication protocols such as: SATA (Serial ATA) – Used in traditional SSDs. NVMe (Non-Volatile Memory Express) over PCIe – Used in high-performance SSDs. USB – Used in flash drives.

Storage Technologies

SSDs-Architecture



Basic Solid State Drive (SSD) Architecture

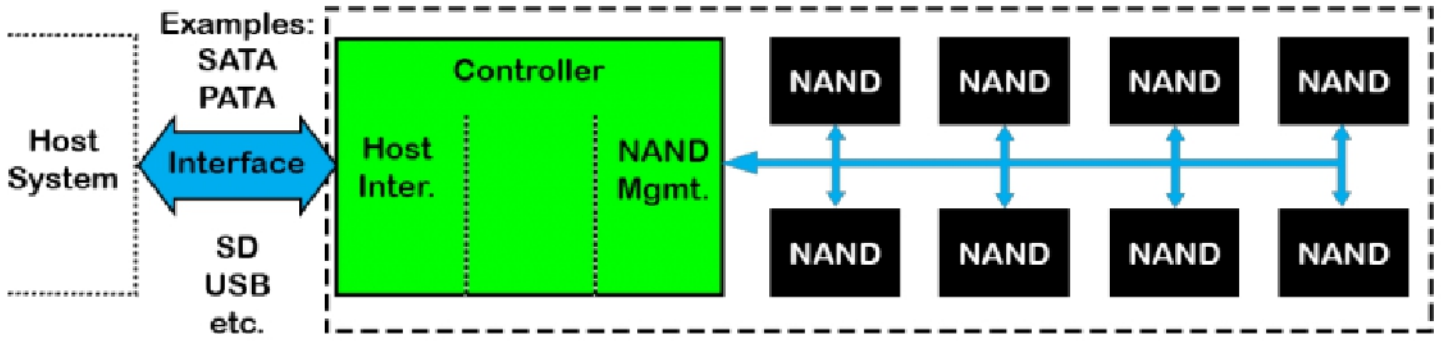
Controller chip

- The Controller is the "brain" of the NAND flash memory system. It manages how data is read, written, and erased efficiently while ensuring reliability and longevity. The controller consists of two main components:

1. Host Interface (Host Inter.): This part of the controller handles communication with the external device. It translates data requests from the host system (computer, phone, embedded system, etc.) into commands that the NAND flash memory can understand. It Manages data transfer speeds and ensures compatibility with different communication protocols (SATA, NVMe, USB, etc.)

Storage Technologies

SSDs-Architecture



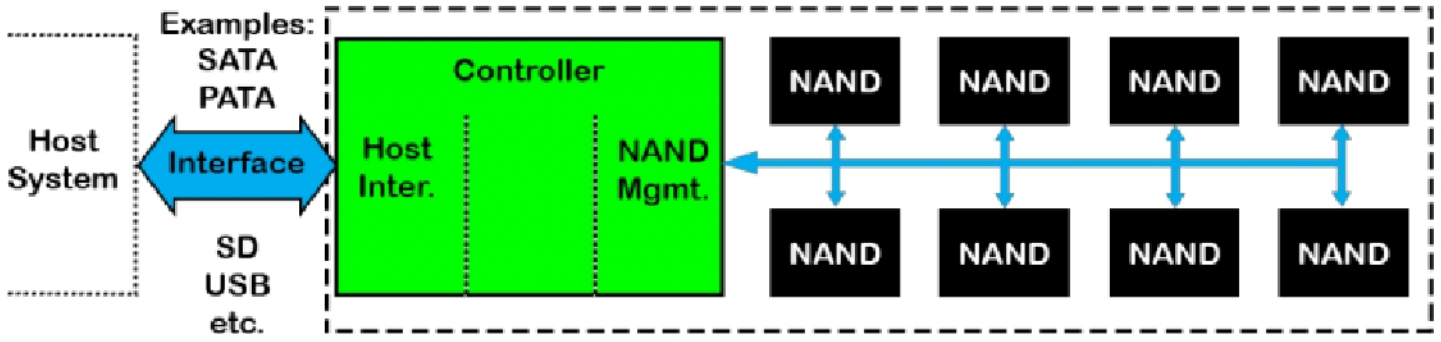
Basic Solid State Drive (SSD) Architecture

2. NAND Management

- Manages the internal operations of NAND flash memory, ensuring reliable and efficient data storage. Key responsibilities include:
  - ✓ Wear Leveling – Distributes write operations evenly across the NAND blocks to prevent premature wear.
  - ✓ Bad Block Management – Detects and avoids faulty memory blocks.
  - ✓ Error Correction Code (ECC) – Detects and corrects data corruption due to charge leakage in NAND cells.
  - ✓ Garbage Collection – Reclaims erased blocks for future use to maintain performance.
  - ✓ Translation Layer (FTL - Flash Translation Layer) – Maps logical addresses (used by the OS) to physical addresses (in NAND).

Storage Technologies

SSDs-Architecture



Basic Solid State Drive (SSD) Architecture

NAND Flash Memory Array

- The NAND flash memory chips store actual data.
- They are connected in a parallel or serial arrangement to optimize performance and capacity.
- The memory is organized into multiple levels:
  Pages – The smallest unit of data storage (usually 4KB to 16KB).
  Blocks – A collection of pages (e.g., 128 to 256 pages per block).
  Planes – A group of blocks that can be accessed simultaneously to improve performance.
  Dies – Multiple planes inside a single NAND chip.
- The NAND chips are connected through multiple channels to allow parallel read/write operations, enhancing speed.

Storage Technologies

SSDs-Architecture

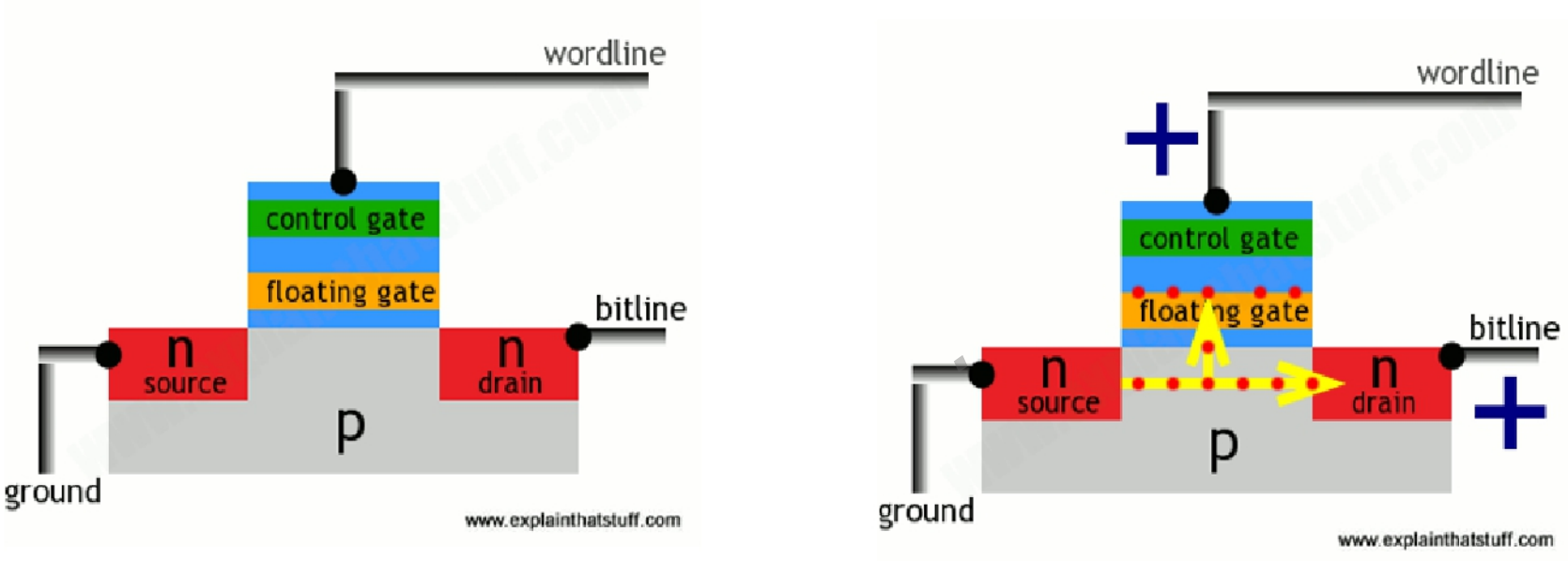


Fig: NAND Flash Cell

Storage Technologies

SSDs-Architecture

- Each NAND Flash memory cell is made up of tiny transistors that store data.
- Each transistor has three main parts:
    - Source (starting point for electrons)
    - Drain (ending point for electrons)
    - Floating Gate (a layer that traps electrons to store data)

- Two important wires control the transistor:
    - Wordline (controls the floating gate)
    - Bitline (controls the flow of current)
- A normal transistor has three connections: source, drain, and gate. The gate control the flow of current between source and drain. When the gate is closed (transistor is off), storing a zero. When the gate is open, electricity flows through (transistor is on), storing a one. When power is turned off, the transistor resets and "forgets" the stored value.
- But, flash memory uses a special floating gate transistor with two gates: a control gate and a floating gate. The floating gate is isolated and can hold electric charge even when the power is off. When electricity leaks into the floating gate (while the gate is open), it traps charge between the two gates.

Storage Technologies

SSDs-Architecture

- When charge is trapped in the floating gate, it blocks the current from flowing, and the transistor stores a zero.
- When the charge is cleared from the floating gate, current can flow, and the transistor stores a one.
- Single Level Cell NAND type can store single bit per cell similarly multilevel can store two bit and three level cell NAND can store 3 bit per cell.
- The floating gate holds electrons even when power is turned off. The floating gate is completely surrounded by a layer of silicon dioxide (SiO_2). Silicon dioxide is an excellent electrical insulator, which means electrons cannot escape easily. So, Once electrons are trapped inside the floating gate, they remain there even when power is turned off. This means the data stays even without electricity, making flash memory non-volatile.

Writing Data (0)

- We apply high voltage to the wordline and bitline.
- This allows electrons (negatively charged) to flow from source to drain.
- Some electrons jump through an insulating layer and get trapped in the floating gate.
- These trapped electrons block the current from flowing later. This negative charge repels the electrons in the channel, preventing them from moving from source to drain.
- When current is blocked, the transistor stores a '0'.

Storage Technologies

SSDs-Architecture

Erasing Data (Storing a '1')

- To remove the stored '0', we apply a negative voltage to the wordline.
- This pushes the trapped electrons out of the floating gate.
- Now, the current can flow freely again.
- When current flows, the transistor stores a '1'.

Reading Data

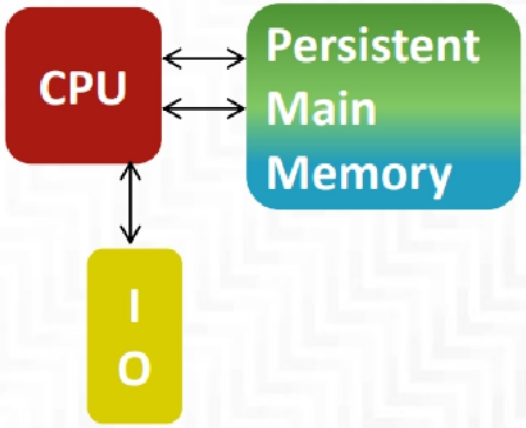
- We apply a small voltage to the wordline.
- If current flows, the transistor is storing a '1'.
- If no current flows, the transistor is storing a '0'.
- This is how the system knows whether the memory cell is storing 1 or 0.

Storage Technologies

Persistent Memory.

- Persistent Memory (PM) is a special type of memory that combines the speed of RAM with the persistence of storage.
- It allows data to remain even after power loss, but operates much faster than traditional NVM (like SSDs).
- Persistent memory updates modern computing architecture to include main memory where stored data is preserved even when power is not applied.
- It blurs the line between memory and storage, creating a new class of hardware and disrupting the platform paradigm of most software
- The core of PMEM is built using advanced non-volatile memory materials such as:

✓ 3D XPoint (Intel Optane) → Fast, durable, and non-volatile.
✓ Resistive RAM (ReRAM) → Uses resistance changes to store data.
✓ Magnetoresistive RAM (MRAM) → Uses magnetic storage principles.
✓ Phase-Change Memory (PCM) → Uses heat to change the state of material.



Storage Technologies

Persistent Memory: Features

- **Byte Addressable Access:** Unlike SSDs (which require block-based access), PMEM allows CPU to access data at the byte level, just like DRAM. This makes read/write operations much faster and more efficient.
- **Non-Volatile Storage:** PMEM retains data even when power is turned off, unlike DRAM, which loses data instantly. It does this using advanced memory technologies such as 3D XPoint, ReRAM, or MRAM.
- **Direct Memory Access:** In DAX mode, PMEM allows applications to access it directly as memory, without needing a file system. This removes storage overhead and speeds up performance.
- **Memory Mapped I/O:** PMEM can be mapped into a system's memory space, allowing the CPU to read/write data just like DRAM.
- **Durability and High Endurance:** Unlike NAND flash, PMEM has better endurance (i.e., more write cycles before wear out). Some PMEM technologies use resistive or phase-change storage mechanisms, which are more durable than flash.

Further Study

- Modern File Systems: EXT4 and APFS
- Distributed File System: Ceph
- FS/Volume Model vs ZFS.
- HDD vs SDD
- Persistent Memory Vs Non-Volatile Memory
- Case Studies: DNA Storage, Holographic Storage, Quantum Storage